

Rasaoi Staging Handoff

Simple guide for stakeholders · Ticket **ROE-016** (Experimental staging) · 31 July 2026

Bottom line: A safe *practice* website is live so we can try big new features without touching the real customer site. Production is unchanged. Staging Ask → Reading was checked and works.

Staging website	https://v0-rasaoi-staging.vercel.app
Real (production) site	https://rasaoi-delta.vercel.app — not changed by this work
Staging data home	Separate Supabase project (aot1zhdgnvovvqxmggyx)
Latest ship	Commit cd6be34 · Vercel deploy succeeded (31 Jul 2026)
Not ready yet	Merge into main product branch (deve1op) — waiting on a final plate soak + your OK

1. Why we built staging

Rasaoi ranks restaurants and builds meal “plates” from real menus. New AI and data features are powerful — and risky if they invent dishes or break the live app.

Staging is a **copy playground**: same kind of app, separate servers, separate data. We turn new features **on** there first. Production stays on the proven path until you approve.

2. Advantage comparison (old vs staging)

Plain language: what production does today vs what staging adds, and why it helps.

Feature	Before (production today)	After (staging now)	Advantage for Rasaoi
Safe place to try changes	Most tests hit the same live path or local machines.	Own website + own database. Flags on only for staging.	We can break things safely. Customers never feel experiments.
Which AI model answers Ask	Always Gemini only.	Smart router can try other models; if they fail, Gemini still answers.	Less lock-in to one vendor. Better uptime. Room to pick cheaper/faster models later.
Dish knowledge for ranking	Fixed file built offline (culinary-index.json). Slow to refresh.	Live database overlay of dish knowledge on top of that file.	Menus and nutrition facts can update without a full rebuild. Ranking stays richer.

Feature	Before (production today)	After (staging now)	Advantage for Rasaoi
Nutrition / blood-sugar help	Heuristics + Gemini estimates. Gaps can be guessed.	3-step food breakdown; unknown bits go to a <i>quarantine</i> (held aside). Linked into the GL lens.	Honest about missing data. Less fake nutrition. Safer for wellness claims.
Learning from user feedback	Check-ins are write-only. Hard for tools to read safely.	Secure mirror + script turns weak ratings into written “don’t do this again” rules.	Closed loop: real user pain becomes automated test cases.
Stress-test for bad / weird Asks	Small manual checks.	520 chaotic test Asks; simulator scored 100% (goal was $\geq 98\%$).	Catch invented dishes and regional mistakes before they hit users.
Keeping menus fresh	Lab / Firecrawl / manual ingest.	Weekly Apify job + webhook; knowledge updates first. Live menu publish needs a human promote step.	Automation without silent menu inventing. Catalog stays trustworthy.
Secrets & keys	Prod env only.	Separate <code>.env.experimental</code> + staging secrets (USDA, OpenAI, Anthropic, Apify...).	Keys never mix with production. Easier to revoke staging-only access.
Ship process	Prod CI on develop / main.	Dedicated GitHub Action deploys staging branch → v0-rasaoi-staging.vercel.app .	Every staging push can auto-update the practice site.
Guardrails policy	Pairings rules in code + tests.	Same rules + standing “no invented dishes” Cursor policy + impact audits.	Agents and humans share one clear “do not invent plates” contract.

3. Features live on staging (detail)

EXP-T8 Dedicated staging site

What: <https://v0-rasaoi-staging.vercel.app> talks only to staging Supabase.

Advantage: Zero risk to rasaoi-delta. Stakeholders can click the same Ask → Reading flow.

EXP-T2 / T10 **Model router**

What: Ask, glycemic, and menu ingest go through a router. Gemini is the safety net.

Advantage: We can trial OpenAI/Anthropic on staging without stranding users if a key fails.

EXP-T1 **Dynamic culinary overlay**

What: Reading loads dish knowledge from staging Postgres and overlays the static index.

Advantage: Fresher dish facts. If remote load fails, the old static file still works.

EXP-T3 **Nutrition quarantine + blood-sugar lens bind**

What: Unmapped ingredients are stored as quarantine rows, not invented macros. Lens can use verified binds.

Advantage: Better honesty for “healthy / GL” asks. No silent fake calories.

EXP-T5 **Telemetry → guardrails**

What: Check-ins mirror to an experimental feedback table. A script can feed low scores into `negative_guardrails.xml`.

Advantage: Bad outcomes become permanent regression tests.

EXP-T4 / T9 **Adversarial simulator (520 seeds)**

What: Automated “chaotic Ask” corpus. Gate passed at **100%** (target $\geq 98\%$).

Advantage: Proof we resist inventing Dal Tadka on South Indian menus, carrier-only plates, etc. (heuristic gate — not the same as live LLM QA).

EXP-T7 / T11 **Apify menu pipeline + weekly cron**

What: Webhook + Actor schedule Sunday 06:00 UTC. Writes *knowledge* first. Promoting into live `menu_items` needs an explicit promote command.

Advantage: Menus can refresh automatically while plates still come only from approved catalog dishes.

EXP-T6 **Isolation**

What: Experimental migrations and secrets stay off production. Docker is optional; staging uses remote Supabase.

Advantage: Clear wall between experiment and money path.

4. What we did for this deployment (action log)

Detailed list of work that got staging to today's state (including the ship of commit `cd6be34`).

A. Foundation

1. Opened ticket ROE-016 (EXP-001) with impact analysis and hallucination-guard docs.
2. Created sandboxed code under `src/lib/experimental/`, `scripts/experimental/`, `migrations_experimental/`.
3. Provisioned staging Supabase project and applied core + experimental schema (vector knowledge, RLS, later quarantine + feedback RPCs).
4. Backfilled ~772 culinary knowledge dishes; seeded ~18 Folsom/EDH Indian restaurants / ~642 dishes.
5. Verified Reading data path with `verify-reading` (PASS).

B. Product features on staging

1. Built model router (edge) with Gemini fallback; wired into parse-intent, estimate-glycemic, ingest-menu.
2. Turned on dynamic culinary hydrate + overlay on Reading when Vite experimental flags are true.
3. Added nutrition deconstruction CLI + quarantine persist + glycemic lens adapter.
4. Mirrored outcome check-ins into experimental feedback; added telemetry→guardrails script.
5. Expanded adversarial corpus to 520 seeds; simulator GATE PASS 100%.

6. Built menu export / mirror / scrape / promote scripts; extended Apify webhook for batch payloads.
7. Pushed Apify Actor and weekly schedule `rasaoi-weekly-menu-sync`.

C. Hosting & CI

1. Created dedicated Vercel staging project (public URL: <https://v0-rasaoi-staging.vercel.app>) — not the production project.
2. Added GitHub Action “Deploy staging preview” with staging Vite env + `VERCEL_STAGING_PROJECT_ID`.
3. Fixed staging CI / Vite framework settings so deploys succeed.
4. Set staging edge secrets (Gemini; optional OpenAI/Anthropic/Apify/USDA via experimental env).
5. Deployed edge functions including `experimental-apify-webhook`.

D. Quality & safety

1. Standing Cursor rules: context-guard, hallucination-guard, ROE ticket flow.
2. Blocked speculative fuzzy dish matches across restaurants (G-01).
3. Kept dish-non-invention: plates still from menu / matrix, not from LLM text alone.
4. Ran Vitest (including experimental suite); Ask → Reading smoke confirmed by stakeholder.
5. Synced plan files, `project.md`, CURSOR guides, TODO / impact docs.

E. This latest ship (31 Jul 2026)

1. Committed `cd6be34` on branch `staging` (nutrition, telemetry, corpus, Apify, docs).
2. Pushed to `origin/staging`.
3. GitHub Action run succeeded → Vercel updated <https://v0-rasaoi-staging.vercel.app>.

5. What staging is *not* (important)

- Not production. Do not treat staging data as customer truth.
- Apify cron does **not** auto-publish scraped dishes onto live plates. Promote is manual on purpose.
- Simulator 100% is a **heuristic** gate. We still want a short human “plate soak” (oceany / sweet / Jain / South Indian) before merge.
- No merge to `develop` until you approve after that soak.

Ask of stakeholders: Spend 10–15 minutes on the staging site with a few real Asks (oceany, something sweet, Jain/birthday, South Indian). Confirm plates look menu-true. Then say **Pass** or note issues. After Pass, engineering can open the develop PR.

6. Quick links

- Staging app: <https://v0-rasaoi-staging.vercel.app>
- Production app: <https://rasaoi-delta.vercel.app>
- This handoff (GitHub Pages): <https://abhiritz.github.io/rasaoi-outcome-engine/>
- PDF download: [handoff PDF](#)
- Deploy run (latest): GitHub Actions → Deploy staging preview on staging
- Deep tech notes: docs/impact_analysis_experimental_infra.md, docs/ROE-016-hallucination-guard-impact-analysis.md
- Ticket board: docs/experimental/tickets/README.md

Rasaoi Outcome Engine · ROE-016 / EXP-001 staging handoff · Prepared 31 July 2026 · Language kept simple for cross-team stakeholders · Status: staging **LIVE** · develop merge **WAITING ON SOAK**